Adalat AI

Product Specification, Technical Architecture, and Developer Implementation Guide

Version: 1.1 **Date:** 18 October 2026 **Product:** Text-first AI courtroom simulation and legal practice platform for Indian law
Audience: AI engineer / full-stack developer building the first working version

1. Executive summary

Adalat AI is an educational legal-practice platform. A user chooses a public case, creates a fictional training case, or uploads private case materials. They prepare the case with legal research, evidence organization, argument planning, and witness preparation. They then enter a text-based courtroom simulation where the user plays exactly one side—prosecution or defense—while AI agents fill the other courtroom roles. Each hearing is time-limited. When the session timer expires, Adalat AI safely closes that hearing segment, preserves the trial context, records a simulated adjournment / next hearing date, and generates a session report. The user may resume immediately or return on the suggested date. A private, opt-in AI coach called **Nyaya Saathi** can suggest what the user might say next without speaking on the user's behalf. After each hearing, Adalat AI produces a transcript, structured feedback, and a performance report.

The product must simulate a **controlled courtroom process**, not just a group chat between role-playing bots. A deterministic trial controller owns the hearing state, permitted actions, turn order, objections, and end conditions. LLM agents provide role-specific analysis and language, but do not get unrestricted authority to change trial state or invent legal rules.

Product goals

1. Make repeated courtroom practice available to law students and, later, practicing lawyers.
2. Ground legal research in traceable legal sources and citations.
3. Let users rehearse arguments against a challenging opposing counsel.
4. Support witness examination using defined witness profiles and case facts.
5. Give useful, evidence-based feedback after each trial.
6. Keep uploaded case materials private and isolated from public legal content and other users.

Non-goals for the first release

- Replacing a lawyer, judge, or real legal advice.
- Predicting a real court's outcome with certainty.
- Filing cases or taking actions in real legal systems.
- Fully autonomous, unbounded agent-to-agent conversations.
- Real-time voice, avatars with lip-sync, or video courtroom rendering.
- Allowing the coaching assistant to submit arguments for the user automatically.
- Ingesting every Indian legal document before an MVP can launch.

2. Product scope and assumptions

Initial product assumptions

- **Jurisdiction:** India.
- **Initial UI:** text-based streaming courtroom.
- **Initial users:** law students; design data models so judiciary aspirants and lawyers can be added.
- **Case sources:** public judgments / legislation, user uploads, and fictional educational cases.
- **User role:** exactly one active advocacy role per trial: prosecution or defense.
- **Other roles:** Judge AI, opposing counsel AI, and optional Witness AI.
- **Frontend:** Next.js, React, TypeScript.
- **Backend:** Python, FastAPI, Pydantic.
- **Workflow orchestration:** LangGraph.
- **LLM / retrieval integrations:** LangChain where useful.
- **Vector search:** Qdrant.
- **Relational database:** Neon-hosted PostgreSQL.
- **Web research:** Firecrawl search / scrape / crawl workflows, with source verification.
- **Initial interaction:** text events streamed to the browser.

These are design defaults, not claims that a specific vendor or model is the only correct choice.

Success criteria for the MVP

- A user can sign up, edit profile preferences, create a case workspace, prepare a case, start a trial, complete a controlled hearing, and open the saved transcript/report.
- Every material legal research claim shown as authoritative has a traceable source, and the UI distinguishes retrieved evidence from verified legal authority.
- Agent turns obey the trial state machine.
- The user cannot access another user's private case files by changing an ID in a request.
- A trial can resume after a transient error without duplicating already-committed events.
- Each hearing has a configurable time limit; expiry triggers a safe checkpoint, session report, and simulated adjournment without losing case context.
- Users can start the next hearing immediately or later, continuing from the saved previous-hearing context.
- The optional coaching assistant returns private, case-grounded suggestions and never submits them as the user's words.
- Evaluation measures legal citation quality, role adherence, procedural correctness, latency, and user-rated usefulness.

3. Users and core jobs

Law student

Wants to understand facts, applicable provisions, arguments, courtroom procedure, and how to respond to questions. Needs guided practice, hints, and feedback.

Judiciary aspirant (later / optional in MVP)

Wants issue identification, legal analysis, procedural understanding, and reasoned judicial decision practice. This should eventually be a separate **judge-training mode**, not a confusing variation of advocate mode.

Practicing lawyer (later / advanced)

Wants to stress-test a case theory, anticipate the opposing side, prepare witness questions, and rehearse a hearing. Needs stronger privacy controls and explicit warnings not to treat AI output as verified legal advice.

Profile and preferences

The dashboard includes a user avatar. Clicking it opens profile/settings where the user can edit display name, education / role, experience level, preferred language, learning goals, preferred difficulty, and accessibility preferences. Store only useful preferences. Use them to personalize hints, feedback depth, difficulty, and UI defaults—not to alter the legal merits of a case or silently change agent decisions.

4. Information architecture and user journey

Main routes

- `/` — landing page, product explanation, demo, sign-up CTA.
- `/login` and `/signup` — authentication.
- `/onboarding` — role, learning level, language, goals, and optional preferences.
- `/dashboard` — recent trials, performance trend, saved workspaces, public case search, “New case”.
- `/cases` — searchable public case library and fictional training scenarios.
- `/cases/new` — create a case by public-case selection, upload, or fictional scenario.
- `/cases/[caseId]` — case workspace and preparation.
- `/cases/[caseId]/research` — legal research, citations, source verification.
- `/cases/[caseId]/evidence` — evidence inventory and provenance.
- `/cases/[caseId]/witnesses` — witness profiles and question plans.
- `/trials/[trialId]/setup` — choose side, difficulty, simulation settings.
- `/trials/[trialId]/courtroom` — text-first courtroom UI and event stream.
- `/trials/[trialId]/report` — transcript, scorecard, feedback, replay.
- `/profile` — profile/preferences (can also be a popover from the avatar).

End-to-end flow

1. **Landing:** explain practice, research grounding, privacy, and educational limitations.
2. **Authentication:** use a maintained auth provider or a secure session implementation. Do not roll custom password storage casually.
3. **Onboarding:** ask only for fields that materially improve the experience. Preferences are editable later.
4. **Dashboard:** show recent trials, continue preparation, performance trend, public case search, and create-case actions.
5. **Choose source:** public judgment/case, private upload, or fictional case.
6. **Create workspace:** normalize case title, jurisdiction, court, dates, legal area, parties/roles, case summary, and document inventory. Keep unknown facts explicitly unknown.
7. **Prepare:** parse files, extract text, build evidence inventory, research statutes/judgments, record citations, draft case theory, prepare arguments, and create witness plans.
8. **Review and freeze a trial snapshot:** the user confirms the facts/documents available to the simulation. Save a versioned snapshot so later edits do not silently rewrite a trial in progress.
9. **Trial setup:** choose prosecution or defense, difficulty, hearing format, and whether witness examination is enabled.
10. **Courtroom:** run the state machine; stream typed events to the UI. User input is accepted only when the user has the floor or a permitted action is available.
11. **Close hearing:** judge produces a simulated ruling / disposition appropriate to the exercise, with reasons and limitations. Do not represent it as an actual court judgment.

12. **Report:** show transcript, legal argument quality, evidence use, response to objections, procedural conduct, citation quality, and next exercises.
13. **Replay:** let the user retry with hints, a different difficulty, or a different side without corrupting the original trial record.

Dashboard design

Recommended dashboard sections:

- Greeting and profile avatar (top-right).
- “Continue trial / preparation” cards.
- “Start a new practice” primary action.
- Search/filter public cases by court, legal area, year, keywords, and case type.
- Past trials/hearings with date, case title, side played, status, time used, latest hearing number, next suggested hearing date, session score, and overall progress.
- Performance chart over completed trials, with rubric dimensions available on hover/click.
- Saved research reports and drafts.
- Privacy indicator for private uploads.

Do not use a single composite score as the only feedback. Show the underlying dimensions and explain how scores were produced.

5. Case data model and source separation

Treat a case workspace as a container of **facts, documents, legal authorities, user analysis, and trial versions**. These are different data types.

Source classes

1. **Legal authority:** legislation, rules, judgments, official circulars, and other legal sources.
2. **Case record / evidence:** user-uploaded pleadings, witness statements, exhibits, reports, and other case documents.
3. **User work product:** case theory, notes, arguments, questions, draft submissions, and research report.
4. **Simulation artifacts:** trial events, agent outputs, rulings, scores, and transcript.

Each source must have provenance: where it came from, when it was retrieved/uploaded, who owns it, whether extraction was successful, and whether it has been verified.

Important separation rule

A statement in a user-uploaded document is not automatically true. A search result is not automatically authoritative. An LLM summary is not a primary source. Preserve these distinctions in the data model and UI.

Suggested relational entities

- `users`: identity reference, profile, created/updated timestamps.
- `user_preferences`: role, experience, language, difficulty, learning goals, accessibility settings.
- `cases`: owner, title, source type, jurisdiction, case type, status, privacy scope.
- `case_documents`: storage key, checksum, MIME type, original filename, owner/case ID, processing status.
- `document_passages`: document ID, page/paragraph locator, extracted text, metadata.
- `legal_sources`: source URL, publisher/court, title, jurisdiction, source type, publication/decision date, retrieval time, verification status, checksum.

- `case_facts`: fact statement, source passage, status (alleged/admitted/disputed/unknown), created by.
- `evidence_items`: type, description, source document, provenance, side relevance, objections/limitations.
- `witness_profiles`: name/label, role, statement source, known facts, unknown facts, permitted knowledge boundary.
- `research_runs`: query, filters, sources found, citations, verification status, timestamp.
- `case_notes` / `case_theories`: user-owned analysis and versions.
- `trials`: case snapshot ID, user side, difficulty, status, started/ended times, graph thread ID; represents the full practice case / proceeding across multiple hearings.
- `hearing_sessions`: trial ID, session number, configured duration, start/end/expiry timestamps, status, opening context snapshot, closing checkpoint, adjournment reason, suggested next hearing date, actual resume time. One trial can contain multiple hearing sessions.
- `session_reports`: hearing session ID, rubric version, session-specific scores, observed strengths, improvement points, missed opportunities, generated time.
- `coach_requests`: trial/session ID, user prompt/context event ID, selected suggestion mode, returned suggestions, citations/evidence IDs, timestamp. Store minimally and do not expose private coaching content to opponent agents.
- `trial_events`: ordered event ID, trial ID, sequence number, event type, actor, structured payload, timestamp, idempotency key.
- `trial_transcripts`: rendered transcript or derive it from events; if stored, keep the event log as canonical.
- `trial_reports`: overall rubric version, scores, explanations, evidence links, generated time.
- `hearing_sessions`: trial ID, session number, configured duration, start/end/expiry timestamps, status, closing checkpoint, suggested next date.
- `session_reports`: hearing ID, session-level rubric scores, strengths, improvement items, event references, generated time.
- `coach_requests`: minimal metadata for optional coaching usage and evaluation; do not expose private coach messages to other agents.
- `audit_events`: security-sensitive actions, access changes, export/delete events.

Use UUIDs, foreign keys, timestamps, constraints, indexes, and migrations (Alembic). Add tenant/owner checks to every case/document/trial query. PostgreSQL row-level security can be a defense-in-depth option, but application authorization must still be correct.

File storage

Do not store large raw PDFs as PostgreSQL rows by default. Store files in private object storage (S3-compatible storage or a managed provider) and store object keys/metadata in PostgreSQL. Use short-lived signed URLs after authorization. Virus-scan uploads where possible, enforce file-size/type limits, and handle OCR failures visibly.

6. RAG dataset strategy

Primary candidate: Open India Law

Dataset: https://huggingface.co/datasets/vaquill/open-india-law Repository: https://github.com/Vaquill-AI/open-india-law

This is a strong candidate for the public-law retrieval corpus because it provides structured Indian legal material from official government sources, including Supreme Court and High Court judgments, tribunal/regulator decisions, and legislation broken down into searchable provisions. The published snapshot is large, so do **not** attempt to ingest every row into a small MVP deployment on day one.

The dataset documentation says district and trial court decisions are not included. It is a snapshot, not a guarantee that every item reflects current law. Verify important sources against official publisher pages before showing them as relied-upon authority.

Recommended ingestion order

Phase A — narrow, high-value corpus

- Relevant central legislation and sections for the first legal domain.
- Selected Supreme Court and High Court judgments.
- Selected tribunal sources only if the domain needs them.
- A curated list of authoritative official sources for live verification.

Phase B — expand by domain

- Add state legislation and relevant High Courts.
- Add additional case types after retrieval quality tests.
- Add tribunal/regulator corpora only where relevant.
- Track source coverage gaps explicitly.

Phase C — scale

- Stream selected Parquet shards/rows, filter before embedding, batch embedding requests, checkpoint progress, and upsert idempotently.
- Never load the full corpus into RAM as one dataframe.
- Maintain corpus snapshot/version and ingestion manifest.

License and attribution

The dataset card identifies the data/compilation as CC BY 4.0 and the scraper code as Apache-2.0. Preserve required attribution and source-publisher metadata. The underlying legal text has its own legal basis and exclusions; do not treat the dataset license as a blanket permission to ignore court publication restrictions, privacy rules, sealed matters, or applicable law. Review the dataset card/license and official source terms before commercial deployment.

Suggested attribution in the product/docs:

> Structured Indian primary-law data from the Open India Law corpus by Vaquill AI (https://github.com/Vaquill-AI/open-india-law), used under CC BY 4.0.

Do not treat the corpus as the user's case file

Keep public law content and private case uploads in separate logical namespaces and collections or enforce strict mandatory filters. Public law retrieval can be shared; private case facts and documents must be scoped to the authorized user/case/trial.

7. Ingestion pipeline

Recommended pipeline:

1. **Select source/config:** record dataset release, config/subset, license, date, and source URL.
2. **Read incrementally:** use Hugging Face `datasets` or Parquet tooling with streaming/row-group processing where appropriate.
3. **Normalize schema:** map source fields to a stable internal document schema.
4. **Validate:** require stable source ID, source URL or traceable publisher reference, title/text, jurisdiction, and source type. Quarantine malformed records.
5. **Deduplicate:** use stable case/act IDs and content hashes. For judgment chunks, preserve case ID and chunk order.
6. **Classify:** legislation, judgment, tribunal order, circular, or other.
7. **Chunk intelligently:** preserve section/subsection boundaries for legislation and paragraph/heading boundaries for judgments. Avoid splitting a statutory subsection arbitrarily. Preserve original paragraph numbers.
8. **Enrich metadata:** jurisdiction, court, bench, date, citation, legislation status, section number, source publisher, source URL, snapshot version, and document type.

9 **Embed in batches:** choose an embedding model and record its exact name/version/dimension. Re-embedding is required if the model or normalization changes.

10 **Upsert to Qdrant:** use deterministic IDs and payload metadata; make retries idempotent.

11 **Create payload indexes:** for fields used in filtering (jurisdiction, court, date/year, source type, act status, case ID, owner/tenant where applicable).

12 **Quality check:** compare indexed counts, sample records, source links, duplicate rate, and retrieval results.

13 **Publish manifest:** corpus version, filters, counts, failures, model version, and completion time.

14 **Refresh:** schedule incremental refresh and a process to mark repealed/amended provisions and superseded sources.

Suggested normalized legal chunk

```
{
  "chunk_id": "stable-deterministic-id",
  "source_id": "official-source-or-dataset-id",
  "source_type": "legislation_section",
  "title": "Short title / judgment title",
  "text": "Exact passage used for retrieval",
  "jurisdiction": "India",
  "court": null,
  "decision_date": null,
  "act_id": "stable-act-id",
  "section_number": "section-or-paragraph",
  "act_status": "in_force",
  "source_publisher": "Official publisher",
  "source_url": "https://official-source.example/document",
  "corpus_snapshot": "v2026.08",
  "content_hash": "sha256..."
}
```

Treat the above as an internal target schema, not a claim that every source record contains every field. Keep unknown fields null; never fabricate missing metadata.

8. Retrieval and legal research design

Use hybrid retrieval, not embeddings alone

Legal research often depends on exact section numbers, party names, citations, phrases, dates, and legal concepts. Pure semantic retrieval can miss exact identifiers. Recommended flow:

1 **Query classification:** identify whether the user needs legislation, a judgment, a case fact, a procedural rule, or broad research.

2 **Query parsing:** extract jurisdiction, court, date range, act/section, case citation, legal issue, and source requirements.

3 **Search planning:** select Qdrant semantic search plus keyword/sparse retrieval; use metadata filters.

4 **Parallel retrieval:** retrieve from legislation and judgment indexes concurrently when appropriate.

5 **Rank and merge:** use reciprocal rank fusion or another tested fusion strategy; optionally rerank top candidates.

6 **Authority/relevance checks:** prefer primary official sources; assess court level, date, jurisdiction, relevance, current status, and whether a decision has been overruled or superseded when that data is available.

7 **Source verification:** open the official source URL for material claims where possible.

8 **Answer generation:** use only retrieved, source-linked context for legal claims. Separate the holding from the model's explanation.

9 **Citation validation:** each material legal claim should link to a real source and, where available, section/paragraph/page.

10 **Abstention:** if sources are missing, conflicting, stale, or insufficient, state that limitation and request more context rather than inventing a citation.

Firecrawl role

Use Firecrawl as a **discovery and extraction layer**, not as a legal-authority ranking engine.

- Search for relevant official court/government pages.
- Scrape/crawl only allowed and relevant pages.
- Prefer official courts, government portals, and authoritative publishers.
- Keep URL, page title, publisher/domain, retrieval timestamp, page date if present, and a content hash.
- Extract citations and links, then verify high-impact sources.
- Cache results and respect API limits and source terms.
- Treat page content as untrusted input; it must not override system instructions or authorize tools.
- Do not use snippets alone as final legal authority.
- Web search complements the corpus; it does not silently overwrite it.

If Firecrawl does not expose a reliable search result or the source cannot be verified, show it as an unverified lead rather than a confirmed authority.

RAG answer schema

Each research result should include:

- `claim_or_summary`
- `source_title`
- `source_type`
- `publisher_or_court`
- `source_url`
- `date`
- `pinpoint_reference` (section/paragraph/page if known)
- `retrieval_method` (corpus / web)
- `verification_status`
- `relevance_explanation`
- `limitations`

Embedding model choices

- **MVP:** `sentence-transformers/all-MiniLM-L6-v2` is lightweight and a reasonable baseline for prototyping, but evaluate it on Indian legal terminology and exact citation retrieval.
- **Alternatives:** BGE/E5-family models or a strong hosted embedding API; test multilingual needs if Hindi or other Indian languages are in scope.
- Do not pick a model based on general benchmark claims alone. Build a small legal retrieval evaluation set and compare recall@k, MRR/nDCG, citation coverage, latency, and cost.

9. Multi-agent courtroom design

Core roles

Judge AI

- Enforces a role prompt for neutrality and procedural discipline.
- Asks clarifying questions and manages turn transitions.

- Rules on objections only using the configured procedural rules and case context.
- Explains the basis and uncertainty of simulated rulings.
- Does not have unrestricted database writes or control over user permissions.

Prosecutor AI

- Builds a prosecution theory from the case snapshot.
- Presents arguments, uses admissible/available evidence, challenges defense arguments, and conducts prosecution examination.
- Must not invent evidence or misrepresent a cited authority.

Defense AI

- Challenges elements and evidence, identifies inconsistencies, develops alternative explanations, raises applicable objections, and cross-examines within the exercise rules.
- Must not invent facts or claim that an authority says something it does not.

Witness AI (optional per trial)

- Has a bounded witness profile and only the facts the witness could reasonably know.
- Answers questions consistently with the assigned statement and known facts.
- May say “I do not know” or “I do not remember” when appropriate.
- Does not reveal hidden authoring notes or invent new evidence to help either side.

Trial Controller (deterministic application logic, not a free-form agent)

- Owns the current stage, speaker, allowed action, turn count, objection state, time/word limits, and end conditions.
- Validates every transition and persists the event.
- Prevents an agent from skipping procedure or writing arbitrary trial state.

Research service (tool-backed subsystem)

- Retrieves sources and citations.
- Does not itself decide the case outcome.
- Returns structured source records rather than a blob of unsourced text.

Personality vs. role

Give agents distinct courtroom styles (e.g., concise judge, rigorous prosecutor, methodical defense counsel), but separate style from authority. Personality must never change the case facts, legal rules, permitted actions, or evaluation rubric.

Shared case context and private scratch work

All agents receive a consistent, versioned public trial snapshot: agreed facts, disputed facts, evidence metadata, relevant sources, procedural configuration, and current stage. Each role receives only the context appropriate to its job. Do not expose one side's private notes or planned arguments to the opposing agent unless the simulation explicitly models disclosure.

Avoid putting every document and the full transcript into every prompt. Retrieve the most relevant case passages and a compact event summary per turn.

10. Hearing sessions, time limits, adjournment, and continuation

Distinguish a case, a trial, and a hearing session

These should be separate concepts in both the product and database:

- **Case workspace:** the underlying case facts, documents, evidence, research, notes, and witnesses.
- **Trial / proceeding:** the user's complete practice journey for one case, potentially spanning multiple sessions.
- **Hearing session:** one time-limited courtroom meeting within that trial.
- **Session report:** feedback about the user's performance in that one hearing.
- **Overall progress report:** a longitudinal view across all hearing sessions for the same trial.

Do not create a completely new trial whenever a user returns. Create a new `hearing_session` under the existing trial so that the context, previous arguments, rulings, witness statements, and learning history remain connected.

Session timer behavior

Let the user or product configuration choose a duration (for example, 10, 20, or 30 minutes for a training session). The duration should be configurable and stored when the session starts. Do not rely on a browser-only countdown as the authority.

1. The server records `started_at` and `expires_at` using server time.
2. The frontend displays a countdown based on the server timestamps and can warn the user at configured thresholds (for example, 2 minutes and 30 seconds remaining).
3. Each turn/action checks whether the session is still active and whether the requested action is allowed.
4. When the deadline is reached, the server stops accepting new courtroom actions for that hearing session. If an agent turn is in progress, apply a defined policy: cancel it safely or let the current atomic turn finish within a short grace period. Do not allow an unbounded turn to continue.
5. The controller commits a final checkpoint and marks the session `TIME_EXPIRED` / `ADJOURNED` as appropriate.
6. The Judge AI generates a concise simulated adjournment statement and a suggested next hearing date. This date is a training convenience, not a prediction of an actual court listing.
7. The system generates a session report from persisted events and the case snapshot.
8. The user sees two clear actions: **Continue now** or **Resume later**. Continue now creates a new hearing session under the same trial. Resume later leaves the trial in a resumable state and displays the suggested date in the dashboard. The suggested date must not block the user from returning early.

The next hearing must preserve context

At the end of each hearing, create a structured `continuation_checkpoint`, including:

- case snapshot/version ID and any user-approved amendments;
- current procedural stage and next expected actor/action;
- last committed event sequence;
- arguments already made by each side;
- evidence admitted, challenged, or still pending under the simulation's rules;
- objections and rulings;
- witness examination progress and facts already stated;
- unresolved issues, pending submissions, and the reason for adjournment;
- relevant citations/source IDs;
- a compact summary for agent context, clearly marked as a summary rather than the canonical record.

The canonical transcript/event log remains the source of truth. The checkpoint is a compact recovery aid. On resume, load the checkpoint and recent events, validate the trial/session status, and continue from the correct stage. Do not ask an LLM to reconstruct the full state from transcript prose alone.

Time-up report

Generate a **session report automatically when the timer expires**, even if the overall proceeding is incomplete. The report should evaluate only what the user actually did during that session, while using the case context to judge relevance. Include:

- key actions and arguments made;
- strongest argument, with transcript event references;
- missed issues or opportunities;
- legal/source citation quality;
- evidence linkage and factual consistency;
- response to opposing arguments and judicial questions;
- procedural conduct and use of objections, where applicable;
- concise improvement priorities and example next steps;
- unresolved issues to prepare before the next hearing;
- session score by rubric dimension and a short explanation for each score.

Never penalize a user for not completing a stage that was not reached because the timer expired. Separate **performance quality** from **progress/completion**. The report should not invent actions or claim the user failed to do something that the session never gave them an opportunity to do.

The overall trial report should aggregate all session reports carefully. Show per-session trends and rubric dimensions rather than simply averaging incompatible scores. Record the rubric version used for each report.

Timer and concurrency edge cases

- Use a server-side expiry check for every action; a delayed browser timer must not extend the session.
- Make expiry handling idempotent so two workers do not create duplicate adjournment events or reports.
- If the browser disconnects, the timer still expires according to server time.
- If report generation fails, preserve the closed session and enqueue a retry; do not reopen it accidentally.
- If a request started just before expiry, use a transaction/lock or compare-and-set rule to decide whether it was accepted before the deadline.
- Persist a clear `session.status` such as `ACTIVE`, `TIME_EXPIRED`, `ADJOURNED`, `COMPLETED`, or `CANCELLED`.
- Store the suggested next hearing date separately from the actual resume time. Users can resume earlier.
- Use a background scheduler/worker for expiry processing, but still check expiry synchronously in API handlers. The worker is not the only enforcement layer.

Suggested user experience

At the start: “This hearing session lasts 20 minutes. Your work is saved automatically. When time ends, the judge will adjourn the practice hearing and you will receive feedback.”

At expiry: show a simulated judge statement, a “Hearing adjourned” status, the suggested next date, session report status, and **Continue now** / **Resume later** actions. Do not imply that a real court has scheduled anything.

11. AI coaching assistant: Nyaya Saathi

Working name: Nyaya Saathi (“legal companion”). The name can be changed after product/brand review.

Purpose

Nyaya Saathi is a private, optional in-session coach for a learner who is unsure what to say next. It supports learning without taking over the user's role. It is not another courtroom participant and should not appear as a speaker in the public court transcript.

User flow

1. The user opens “Need help?” / “Ask Nyaya Saathi” beside the input composer.
2. The UI sends the current trial/session ID, current stage, the user's selected side, the most recent relevant transcript events, the case snapshot reference, and the user's question (optional).
3. The backend checks ownership, session status, current stage, and coach quota.
4. A coaching workflow retrieves relevant case facts, evidence, legal authorities, and any relevant user preparation notes.
5. The coach returns a small number of actionable suggestions, not a long generic essay.
6. Each suggestion is labeled by purpose, for example:

- **Possible next argument** - **Question to ask** - **Issue to address** - **Potential objection to consider** - **Clarifying question for the judge** - **Research/check this authority**

1. Where a suggestion makes a legal claim, it includes source links/pinpoint references or clearly states that the authority needs verification.
2. The user can copy a suggestion into their composer, edit it, ask for a hint instead of a full draft, or dismiss it.
3. Only after the user explicitly submits the text does it become a courtroom turn. Coaching output is never automatically sent to the judge or opposing counsel.

Coaching modes

Offer a user-controlled assistance level:

- **Hint:** ask a guiding question or point to a relevant issue without writing the answer.
- **Outline:** provide a short structure the user can express in their own words.
- **Draft suggestion:** propose a concise possible response/question, grounded in the case.
- **Research support:** show potentially relevant sources and explain why they may matter.

For learning, default to Hint or Outline. Let users choose Draft Suggestion when they want more direct assistance. A training mode can record that a suggestion was used, with the user's consent, so reports can distinguish unaided performance from assisted performance. Do not automatically lower scores just because help was requested; report assisted and unaided dimensions transparently.

Coach architecture

Implement Nyaya Saathi as a separate LangGraph subgraph or bounded service, called on demand—not as a free-running courtroom agent.

Inputs:

- authorized `trial_id` and `hearing_session_id`;
- user side and current stage from server state (never trust client-provided role as authority);
- last N relevant events plus a compact event summary;
- case snapshot ID;
- optional user question and selected coaching mode;
- retrieval results with source IDs.

Workflow:

1. Validate session ownership and whether coaching is allowed in the current stage.
2. Load the canonical state from PostgreSQL/checkpoint storage.
3. Retrieve only relevant case-specific and public legal passages.
4. Generate structured suggestions with `type`, `suggestion_text`, `reason`, `evidence_ids`, `citation_ids`, and `confidence/limitations`.
5. Validate citation IDs and evidence IDs against retrieved results.
6. Return suggestions privately to the requesting user.
7. Log minimal metadata for observability and optional assisted-performance analysis.

Example response schema:

```
from pydantic import BaseModel, Field
from typing import Literal

class CoachSuggestion(BaseModel):
    kind: Literal[
        "hint", "argument", "question", "objection",
        "issue", "research"
    ]
    text: str
    rationale: str
    evidence_ids: list[str] = Field(default_factory=list)
    citation_ids: list[str] = Field(default_factory=list)
    limitations: list[str] = Field(default_factory=list)
```

The server should validate that referenced IDs exist in the authorized case/research context. If the coach cannot support a legal claim, it should suggest checking the issue rather than inventing an authority.

Keep the coach separate from courtroom agents

- Opposing counsel must not see the user's private coach conversation.
- Judge AI must not see private coaching prompts or draft suggestions unless the user submits them into court.
- Coach output must not change the trial stage, ruling, evidence status, or case facts.
- The coach should not reveal hidden system prompts or private agent scratch work.
- Treat uploaded documents and retrieved web pages as untrusted data; instructions inside them cannot override the coach's rules.
- Apply per-user/session rate limits and cost controls.
- Make coaching optional so users can practice without assistance.

Measuring the assistant's usefulness

Evaluate whether suggestions are:

- relevant to the current stage;
- grounded in the case facts and sources;
- concise and actionable;
- legally plausible and appropriately qualified;
- consistent with the user's assigned side;
- not revealing opposing-side private notes;
- useful to the learner.

Track acceptance/copy/edit/dismissal only if useful and disclosed. User acceptance alone is not proof that a suggestion was correct.

12. Trial state machine

Use a deterministic state machine coordinated by LangGraph. A suggested initial lifecycle:

1. `TRIAL_CREATED`
2. `OPENING_PROSECUTION`
3. `OPENING_DEFENSE`
4. `PROSECUTION_CASE`
5. `WITNESS_EXAMINATION` (optional)
6. `CROSS_EXAMINATION` (optional)
7. `DEFENSE_CASE`
8. `REBUTTAL_OR_REPLY` (only if enabled)
9. `CLOSING_PROSECUTION`
10. `CLOSING_DEFENSE`
11. `JUDICIAL_REVIEW`
12. `SIMULATED_DISPOSITION`
13. `REPORT_GENERATION`
14. `COMPLETED`

This is a configurable educational sequence, not a statement that every real Indian proceeding follows this exact order. Real procedure differs by court, matter type, stage, and applicable law. For MVP, choose one clearly described simplified hearing format and make its limits visible.

Multi-session wrapper around the hearing state machine

The stage machine runs **inside** a hearing session, while the full trial persists across sessions:

```
Trial created from case snapshot
  -> Start hearing session #1 (server sets expires_at)
  -> Run controlled courtroom stages and persist events
  -> Timer expires or session is ended
  -> Freeze session event range + continuation checkpoint
  -> Generate session report
  -> Judge AI gives simulated adjournment + suggested next date
  -> User chooses:
       Continue now -> create hearing session #2 immediately
       Resume later -> leave trial resumable on dashboard
  -> Load prior checkpoint + canonical events
  -> Continue from saved procedural stage
  -> Repeat until the exercise's end condition
  -> Generate overall trial report across all sessions
```

The suggested date is advisory. The system should permit a user to start a new session before that date, subject to product rules. Each new hearing gets a fresh timer but remains attached to the same `trial_id`.

Example turn loop

```
Load trial snapshot
  -> Validate current stage and expected speaker
  -> Build role-specific context
  -> Run current speaker (user input or agent)
  -> Validate structured response
  -> Detect requested objection / action
  -> If objection: pause the argument and ask Judge AI to rule
  -> Validate judge ruling against configured rules
  -> Commit event to PostgreSQL
  -> Update stage / next speaker
  -> Emit event to client
  -> Repeat until end condition
  -> Generate report from persisted event log
```

Objection handling

An objection should be a typed event, not merely text that says “objection.” For example:

- objecting side
- target statement/question/evidence
- objection category
- short grounds
- ruling (`sustained`, `overruled`, `deferred`, or configured alternatives)
- explanation and source/rule reference when available
- next permitted action

The UI and controller enforce the result. If sustained, the simulator may instruct the speaker to rephrase or disregard the material according to the chosen training rules. If overruled, the current speaker can continue. The precise behavior belongs to the configured procedure.

Structured agent output

Use Pydantic schemas, not unconstrained prose, for control decisions:

```
from pydantic import BaseModel, Field
from typing import Literal

class CourtroomTurn(BaseModel):
    speech: str
    intent: Literal[
        "opening", "argument", "question", "answer",
        "objection", "response_to_objection",
        "closing", "procedural_request"
    ]
    proposed_next_action: str | None = None
    citations: list[str] = Field(default_factory=list)
    evidence_ids: list[str] = Field(default_factory=list)
```

This is an illustrative starting schema. The server must validate whether the proposed action is allowed in the current stage; a model output never grants itself permission.

13. LangGraph architecture and parallel work

Graph boundaries

Prefer a few understandable graphs/subgraphs over one huge graph:

- **Case preparation graph:** document processing, facts extraction, legal retrieval, research synthesis, case theory, witness preparation, user review.
- **Trial graph:** stage controller, role turns, objections, witness interactions, stage transition, final disposition.
- **Report graph:** transcript normalization, rubric evaluation, feedback generation, citation/evidence checks.

Where parallelism is useful

Run independent work concurrently when it does not change procedural order:

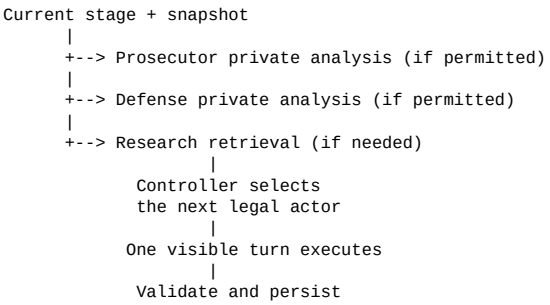
- Parse different uploaded documents in parallel.
- Retrieve legislation and judgments in parallel.
- Ask prosecutor and defense agents for **private preparatory analysis** in parallel before a trial stage, then reveal only the permitted output at the appropriate time.
- Score separate rubric dimensions in parallel after the trial.

- Generate a summary and a citation audit in parallel, then combine.

Where parallelism is unsafe

Do not let Judge, Prosecutor, Defense, and Witness all generate courtroom speech concurrently and then display it as if it happened in real time. That creates contradictory turns, invalid rulings, and incoherent chronology. Actual courtroom actions must be serialized by the trial controller.

Parallel turn preparation pattern



Parallel outputs are drafts or analysis, not courtroom events. They become visible only when the controller permits them.

LangGraph state should contain

- trial/case snapshot IDs and version
- current stage and current actor
- structured current turn
- relevant retrieved passages and source IDs
- objection status
- compact recent event summary
- retry/error state
- completion state

Avoid duplicating large PDFs or all case text in graph state. Persist canonical data in PostgreSQL/object storage and use references. Use a durable checkpointer for production trials; use a local in-memory checkpointer only in development. A stable thread ID should map to a trial session and be authorized on every resume.

Event sourcing and idempotency

Persist each accepted event with a unique sequence number and idempotency key. The trial event log is canonical. If a worker retries after a timeout, the same idempotency key must not create a duplicate turn. A transaction should validate the current stage, append the event, and update the trial state consistently.

14. Text streaming and courtroom UI

Recommended interaction model

Use Server-Sent Events (SSE) for one-way server-to-browser event streaming, plus normal HTTP requests for user actions. WebSockets are an alternative if the product later needs frequent two-way real-time signaling. For a text-first MVP, SSE is simpler.

Example event types:

- `trial.status`
- `agent.started`
- `agent.token` (only if streaming token chunks is supported by the model/provider)
- `agent.message.completed`
- `objection.raised`
- `judge.ruling`
- `witness.answer`
- `citation.attached`
- `trial.stage_changed`
- `trial.error`
- `trial.completed`
- `hearing.warning` (time remaining threshold reached)
- `hearing.expired`
- `hearing.adjourned`
- `session_report.ready`
- `coach.suggestion.ready` (private response to requesting user only)

Every event should have `trial_id`, `hearing_session_id` when relevant, `event_id`, `sequence`, `timestamp`, `actor`, and a typed payload. The server is authoritative for expiry and stage validation. Avoid sending internal reasoning or private agent scratch work to the UI. Stream user-facing speech and safe status updates, not hidden chain-of-thought.

Courtroom layout

- **Left sidebar:** case summary, parties, disputed facts, evidence list, relevant laws, witness list.
- **Center:** courtroom stage, active speaker, transcript bubbles, user input composer, objection/action controls.
- **Right sidebar:** live event feed showing what agents have said, current stage, citations, and procedural events.
- **Header:** case title, role played, difficulty, elapsed time, connection status, pause/end controls.
- **Input area:** text input, send, permitted objection action, attach/select evidence reference if enabled.
- **Mobile fallback:** stacked panels or tabs; desktop is the primary courtroom experience.

The “agent stream” must be an event feed, not an unfiltered dump of internal agent prompts or reasoning.

Reconnection

Clients reconnect using the last event ID/sequence and request missed events. Persist events server-side; do not rely on browser memory as the source of truth. Show a visible reconnecting state and prevent duplicate user submissions.

15. API outline

Use `/api/v1` and validate request/response bodies with Pydantic.

Cases and documents

- `POST /cases` — create workspace.
- `GET /cases` — list only authorized cases.
- `GET /cases/{case_id}` — case detail.

- `POST /cases/{case_id}/documents` — request upload URL or upload document.
- `GET /cases/{case_id}/documents` — list authorized documents.
- `POST /cases/{case_id}/process` — enqueue extraction/ingestion.
- `GET /cases/{case_id}/processing-status` — document pipeline status.
- `POST /cases/{case_id}/research` — start a research run.
- `GET /research/{research_id}` — results and citations.
- `POST /cases/{case_id}/snapshot` — freeze a versioned trial snapshot.

Trials

- `POST /trials` — create trial from a snapshot and user side.
- `GET /trials/{trial_id}` — trial status and allowed actions.
- `POST /trials/{trial_id}/turns` — submit user turn; validate floor/stage.
- `POST /trials/{trial_id}/actions/objection` — submit structured objection.
- `POST /trials/{trial_id}/pause` — pause if supported.
- `POST /trials/{trial_id}/resume` — resume authorized trial.
- `POST /trials/{trial_id}/finish` — request permitted end action.
- `POST /trials/{trial_id}/sessions` — start a new hearing session under an existing trial.
- `GET /trials/{trial_id}/sessions` — list prior and current hearing sessions.
- `GET /trials/{trial_id}/sessions/{session_id}` — status, timer metadata, and continuation checkpoint summary.
- `POST /trials/{trial_id}/sessions/{session_id}/close` — request a controlled close/adjournment where allowed.
- `POST /trials/{trial_id}/coach` — request private Nyaya Saathi assistance for the current turn.
- `GET /trials/{trial_id}/sessions/{session_id}/report` — session-specific report.
- `GET /trials/{trial_id}/events` — paginated/replayable event history.
- `GET /trials/{trial_id}/stream` — SSE stream.
- `GET /trials/{trial_id}/report` — completed report.

User profile

- `GET /me`
- `PATCH /me`
- `GET /me/preferences`
- `PATCH /me/preferences`

Never accept `owner_id` from a client as proof of access. Derive identity from the authenticated session and check ownership/permissions on the server.

Background jobs

Document extraction, OCR, bulk embeddings, long research, and report generation should use a job queue / worker rather than blocking an API request. For a small MVP, choose a simple worker arrangement and add Redis/Celery, Dramatiq, or a managed queue only when needed. The job system is a recommended supporting component, not a mandatory new core platform.

16. Suggested repository structure

```
adalat-ai/
  apps/
    web/                        # Next.js + React + TypeScript
```

```
services/
  api/
    app/
      main.py
      api/v1/
      core/                    # settings, auth, logging
      schemas/                 # Pydantic request/response models
      models/                  # SQLAlchemy models
      repositories/            # database access
      services/                # business logic
      workflows/
        case_preparation/
        trial/
        reporting/
      agents/
        judge.py
        prosecutor.py
        defense.py
        witness.py
      retrieval/
        ingestion/
        retrievers/
        reranking/
        citations/
      tools/
        firecrawl_client.py
      workers/
    alembic/
    tests/
packages/
  shared-types/                # optional generated TS API types
infra/
  docker/
  scripts/
docs/
  architecture.md
  threat-model.md
  evaluation-plan.md
.env.example
README.md
```

Keep agent prompts in versioned files or clearly versioned code constants. Store the prompt/model version used for each trial so results can be reproduced and compared.

17. Recommended stack and alternatives

Area	Recommended first choice	Alternatives / trade-offs
Frontend	Next.js + React + TypeScript	Vite + React if SEO/SSR is unnecessary; Next.js is a good product default
UI styling	Tailwind CSS + accessible component primitives	CSS Modules; use a consistent design system
API	FastAPI + Pydantic	Django REST Framework if admin/ORM-heavy workflows dominate
ORM / migrations	SQLAlchemy 2 + Alembic	SQLModel for a smaller app; raw SQL for performance-critical queries
Relational DB	Neon PostgreSQL	Managed PostgreSQL elsewhere; keep standard Postgres to reduce lock-in
Vector DB	Qdrant	pgvector for simpler infrastructure and smaller scale; evaluate hybrid retrieval requirements
Orchestration	LangGraph	Explicit custom state machine if workflow remains very small; do not build an uncontrolled agent swarm
LLM integration	LangChain integrations where helpful	Provider SDK directly for simpler calls and tighter control
Embeddings	MiniLM baseline for prototyping	BGE/E5 or hosted embeddings after evaluation
Legal web research	Firecrawl plus source verification	Official APIs and direct official-site fetchers; search providers vary in coverage
File storage	Private object storage	Managed upload/storage service; do not expose public buckets
Streaming	SSE	WebSockets for genuinely bidirectional live interaction

Area	Recommended first choice	Alternatives / trade-offs
Background jobs	Simple worker initially	Celery/Dramatiq/managed queue as load grows
Auth	Managed auth provider with secure sessions	Carefully implemented OAuth/OIDC and server sessions
Observability	Structured logs + traces; LangSmith or equivalent	OpenTelemetry + hosted/self-hosted tracing
LLM evaluation	Versioned test set + rubric + LangSmith/Langfuse or custom evaluation	Start with a small hand-reviewed set before adding complex tooling
Deployment	Vercel frontend; managed Python hosting for API/workers	Container hosting / cloud services when workers and throughput need more control

Keep the stack intentionally small

Do not introduce Redis, a graph database, separate microservices for every agent, voice infrastructure, and complex agent protocols before the core text trial works. Add a component only when it solves a measured need.

Agent communication protocol

An agent communication protocol is not required for the first release. Agents can communicate through typed internal messages and shared, permissioned state under the LangGraph controller. Consider ACP or another protocol later if independently deployed agents need interoperable communication across runtimes. Protocol adoption must not bypass the trial controller or source permissions.

18. Security, privacy, and legal safety

Private case material

- Default uploaded case workspaces to private.
- Enforce authorization at every API, storage URL, vector retrieval, export, and background job.
- Separate public legal corpus from private case documents using distinct collections or mandatory tenant/case filters.
- Do not place confidential document text in logs, analytics, or third-party LLM prompts unless the product explicitly discloses and authorizes that processing.
- Provide retention, deletion, and export controls.
- Explain which external providers process data.
- Use encryption in transit and at rest through managed infrastructure.
- Keep API keys in server-side secrets, never in frontend bundles.
- Use upload limits, MIME validation, malware scanning, and OCR sandboxing where feasible.

Prompt injection and untrusted documents

A PDF or webpage can contain instructions such as “ignore previous instructions.” Treat all retrieved text as untrusted evidence, never as system instructions. The model must not gain tools or permissions because a document asks it to. Tool access should be allow-listed, validated, and limited by the current workflow.

Legal content safeguards

- Show jurisdiction, date, source, and verification state.
- Preserve exact quotations separately from paraphrases.
- Distinguish allegations, admitted facts, disputed facts, and established procedural facts.
- Warn that AI-generated research may be incomplete and that important authorities should be checked at the official source.

- Never claim a simulated ruling is a real judgment.
- Avoid uploading or displaying sealed, restricted, or improperly disclosed personal information.
- Add clear educational-use and legal-advice limitations.
- Use a configurable procedural profile rather than claiming the same simplified hearing represents every Indian court.

19. Performance evaluation

A demo is not enough. Build a small benchmark set before expanding the corpus or adding agents.

RAG evaluation

- Recall@k: does retrieval find the relevant source?
- MRR / nDCG: are useful sources ranked near the top?
- Citation correctness: does each link resolve to the cited document?
- Pinpoint correctness: does the paragraph/section support the claim?
- Authority quality: is the source type/court/jurisdiction represented accurately?
- Freshness: are amended/repealed provisions or superseded sources detected?
- Abstention: does the system admit when evidence is insufficient?

Trial evaluation

- Stage/turn compliance.
- Role adherence.
- Factual consistency with the case snapshot.
- Objection handling against the selected rules.
- Citation and evidence use.
- Judge neutrality and consistency.
- Witness consistency and appropriate uncertainty.
- Transcript completeness and event ordering.
- User-rated realism and learning usefulness.

Performance and reliability

- First visible token / event latency.
- Time to complete a turn.
- Retrieval latency and cost per research run.
- LLM cost per trial.
- Failure and retry rates.
- Duplicate event rate (target: zero).
- Unauthorized-access test failures (target: zero).
- Successful resume after interruption.

Scoring rubric

Use an explicit, versioned rubric with dimensions such as:

- issue identification

- legal accuracy and authority use
- evidence linkage
- reasoning quality
- response to counterarguments
- clarity and organization
- procedural conduct
- witness questioning (when enabled)

Scores should include examples from the transcript. Do not let the same LLM that generated a speech assign an unexplained score to itself. Use separate evaluation prompts/models or deterministic checks where possible, and manually review a sample of reports.

20. Build plan: one developer, step by step

Phase 0 — Product and legal scope

1. Choose one initial training format and case domain.
2. Define the exact simplified hearing stages and rules.
3. Decide whether witnesses are included in the first demo.
4. Set default session length options, warning thresholds, expiry/grace policy, and what counts as a session report.
5. Decide whether Nyaya Saathi offers Hint, Outline, Draft Suggestion, and Research Support at launch.
6. Define what constitutes a completed trial, a hearing session, a session report, and an overall report.
7. Write user stories and acceptance tests.

Deliverable: product requirements, trial state diagram, initial rubric.

Phase 1 — Foundation

1. Create a monorepo or two clearly linked apps.
2. Set up Next.js/TypeScript and FastAPI/Python.
3. Add environment configuration, linting, formatting, and tests.
4. Connect Neon PostgreSQL and Alembic migrations.
5. Implement authentication and server-side authorization.
6. Create profile/preferences and dashboard skeleton.
7. Add trial and hearing-session data models, status fields, duration, expiry timestamp, and suggested next date.

Deliverable: user can sign in, edit profile, and see an empty dashboard.

Phase 2 — Case workspace

1. Create case and document schemas.
2. Add private file storage and signed uploads.
3. Implement PDF/text extraction and visible processing states.
4. Save extracted passages and source locations.
5. Build case facts, evidence inventory, and notes.
6. Add fictional starter case for deterministic testing.

Deliverable: a user can create a case and inspect its extracted material.

Phase 3 — Legal RAG

1. Inspect the Open India Law dataset card, configs, schema, sample rows, snapshot and license.
2. Select a narrow subset; do not bulk-ingest the full dataset.
3. Normalize records and preserve original identifiers/source URLs.
4. Implement section-aware/paragraph-aware chunking.
5. Embed and index selected chunks in Qdrant.
6. Add hybrid retrieval, metadata filters, source deduplication and citation schemas.
7. Integrate Firecrawl search/scrape as a separate research source.
8. Build source verification and unverified-source states.
9. Create a small human-reviewed legal query benchmark.

Deliverable: research panel returns useful, source-linked results with honest uncertainty.

Phase 4 — Case preparation

1. Build the preparation graph.
2. Extract candidate facts with document references.
3. Let users mark facts as alleged, admitted, disputed, or unknown.
4. Generate a case theory and argument outline with citations.
5. Generate witness plans only from known source material.
6. Add user review/editing before finalizing the snapshot.
7. Version the snapshot used by each trial.

Deliverable: case preparation produces a saved, editable workspace.

Phase 5 — Trial controller before agent complexity

1. Implement the state machine without LLMs, using a scripted demo.
2. Validate the current speaker and permitted action on the server.
3. Persist events with sequence numbers and idempotency keys.
4. Implement objections and deterministic example rulings.
5. Add pause/resume and trial completion.
6. Test invalid transitions and duplicate requests.
7. Add server-side timer enforcement, idempotent expiry worker, adjournment event, and automatic session-report job.
8. Implement immediate resume and resume-later flows before adding the coach.

Deliverable: a complete simulated trial can run with mock agents and never violate the state machine.

Phase 6 — AI agents

1. Add structured Prosecutor AI output.
2. Add structured Defense AI output.
3. Add Judge AI for questions and configured rulings.
4. Add Witness AI with bounded knowledge and consistency tests.
5. Give each agent only the required context and tools.

6. Add citation/evidence references to structured output.
7. Record prompt/model versions and run evaluations.
8. Add bounded retries and graceful fallback.
9. Build Nyaya Saathi as a separate on-demand coaching subgraph with Hint/Outline/Draft/Research modes, citation validation, private output, and explicit user submission.

Deliverable: agent-generated courtroom turns obey the controller and cite source material where relevant.

Phase 7 — Streaming courtroom UI

1. Build the three-panel layout.
2. Add SSE events and reconnection by sequence/event ID.
3. Show user-facing agent speech and procedural events.
4. Implement the user composer and allowed action controls.
5. Add loading, timeout, error, and reconnecting states.
6. Ensure private scratch work never appears in the event feed.

Deliverable: a user can complete a text-streamed trial from the browser.

Phase 8 — Report and dashboard

1. Generate transcript from canonical events.
2. Run separate rubric evaluation.
3. Link feedback to exact transcript events and sources.
4. Add score history and trend charts.
5. Add replay and “practice this skill again.”
6. Allow user export/delete where appropriate.

Deliverable: each completed trial becomes a useful learning loop.

Phase 9 — Hardening and deployment

1. Add rate limits and quotas.
2. Add structured logs, traces, and cost tracking.
3. Run privacy and cross-tenant access tests.
4. Run prompt-injection and document parsing tests.
5. Test graph resume, worker retry, and event idempotency.
6. Deploy frontend, API, workers, database, storage, and Qdrant with separate secrets.
7. Add backups, retention policy, and incident response basics.

Deliverable: controlled beta with documented limitations and known failure modes.

21. Testing checklist

Functional

- Sign up/login/logout and profile edit work.
- Public case selection and private upload work.

- Extraction failures are visible and recoverable.
- Research citations open the intended source.
- Trial starts only from a valid snapshot.
- Session timer uses server time and expires even if the browser is closed.
- Expiry creates exactly one closing checkpoint, adjournment event, and session report.
- “Continue now” creates a new hearing session under the same trial and restores the prior context.
- “Resume later” preserves the trial and suggested next date without blocking early resumption.
- Nyaya Saathi suggestions remain private until the user submits them as courtroom speech.
- The coach cites valid case/source IDs and cannot change trial state.
- Only the user-selected side is user-controlled.
- Wrong speaker / wrong stage input is rejected.
- Objection events pause the correct action.
- Transcript ordering is stable.
- Report links to real trial events.

Security

- User A cannot retrieve User B's case, document, vector result, trial, report, or signed URL.
- Changing IDs in URLs or payloads does not bypass authorization.
- Private data is not included in public-corpus ingestion.
- Logs do not expose secrets or unnecessary document text.
- Prompt injection in PDF/web content cannot call unauthorized tools.
- Upload type, size, and processing limits are enforced.

Legal RAG

- Search exact section number.
- Search case citation and party name.
- Search a legal concept using different wording.
- Filter by court, jurisdiction, date, and source type.
- Detect unavailable or stale source.
- Distinguish legislation from judgments and secondary commentary.
- Display uncertainty if no reliable result is found.

Agent behavior

- Judge does not advocate for either side.
- Prosecutor and defense do not invent exhibits.
- Witness admits unknown information.
- Agent citations resolve to source records.
- Invalid structured output is rejected or retried safely.
- Agent cannot skip a stage or declare the trial complete unilaterally.

22. Initial environment configuration

Example `.env.example` (placeholders only; never commit real secrets):

```
APP_ENV=development
DATABASE_URL=postgresql+psycopg://USER:PASSWORD@HOST/DBNAME
QDRANT_URL=https://YOUR-QDRANT-ENDPOINT
QDRANT_API_KEY=replace-me
FIRECRAWL_API_KEY=replace-me
LLM_PROVIDER=replace-me
LLM_API_KEY=replace-me
EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2
PRIVATE_OBJECT_STORAGE_BUCKET=replace-me
AUTH_SECRET=replace-with-a-long-random-secret
LANGSMITH_TRACING=false
LANGSMITH_API_KEY=replace-me
```

Use a secret manager in deployment. Never expose server-only variables with a public frontend prefix. Keep separate development, staging, and production credentials.

23. Practical implementation decisions

Start with one well-defined case format

A criminal case can be a useful demonstration because it supports prosecution, defense, evidence, witness examination, and objections. However, the first release must explicitly identify itself as a simplified training simulation and not as a universal representation of Indian criminal procedure. A focused civil or moot-court format is also valid if that better matches the initial audience.

Do not fine-tune first

Start with retrieval, strong source metadata, constrained prompts, structured output, and evaluations. Fine-tuning should only be considered after a measured failure pattern shows that prompting/retrieval is insufficient and a lawful, high-quality training set is available.

Do not make every agent autonomous

Agents should be role-specific workers inside an orchestrated application. Use deterministic transitions for permissions and procedure; use LLMs for analysis and language. Parallelize research and private preparation, but serialize visible courtroom events.

Keep reports auditable

A score should point to transcript moments, evidence, rubric criteria, and source links. Store the rubric version and model version used. A report without traceable reasons will not build user trust.

24. Recommended MVP definition

The first public demo should include:

- Authentication and editable user profile/preferences.
- Dashboard with public case search, recent trials, and basic performance history.
- One focused set of public legal material plus one fictional starter case.
- Private case upload and text/PDF extraction with ownership checks.
- Research panel using Qdrant and Firecrawl with source links and verification states.
- Case preparation workspace with facts, evidence, arguments, and witness notes.
- A versioned trial snapshot.
- Text-based courtroom with user choosing prosecution or defense.
- Judge AI, opposing counsel AI, and optional witness agent.

- LangGraph-controlled stage machine, typed events, objection handling, persisted transcript.
- Automatic session report on timer expiry, including improvement suggestions tied to transcript events.
- Simulated adjournment with a suggested next hearing date and “Continue now” / “Resume later” options.
- Multi-hearing continuation under the same trial with a saved context checkpoint.
- Optional Nyaya Saathi coaching panel with private, source-grounded next-step suggestions.
- Overall report across hearing sessions with per-session trends.
- Basic observability, privacy safeguards, and a regression test suite.

Defer until after MVP

- Full corpus ingestion.
- Real-time voice and speech recognition.
- Multiple court types and every procedural variant.
- Independent agent services and agent-to-agent protocol adoption.
- Complex knowledge graphs.
- Mobile-native app.
- Automated real-world legal actions.
- Claims of predicting case outcomes.

25. Reference links

Dataset and law corpus

- Open India Law dataset: https://huggingface.co/datasets/vaquill/open-india-law
- Open India Law repository and ingestion notes: https://github.com/Vaquill-AI/open-india-law
- Dataset license: https://github.com/Vaquill-AI/open-india-law/blob/main/data/LICENSE.md

Frameworks and infrastructure

- LangGraph: https://docs.langchain.com/oss/python/langgraph/overview
- LangChain Python docs: https://docs.langchain.com/oss/python/learn
- Qdrant documentation: https://qdrant.tech/documentation/
- Qdrant multitenancy: https://qdrant.tech/documentation/manage-data/multitenancy/
- Firecrawl documentation: https://docs.firecrawl.dev/
- FastAPI documentation: https://fastapi.tiangolo.com/
- Next.js documentation: https://nextjs.org/docs
- Neon documentation: https://neon.tech/docs
- Hugging Face Datasets: https://huggingface.co/docs/datasets/

26. Final architecture summary

```
Next.js browser
  |  HTTPS requests + SSE event stream
  v
FastAPI application
  |-- Authentication / authorization
  |-- Case and document services
  |-- Research API
```

```
  |-- Trial controller / event validation
  |-- Report API
  |
  +--> PostgreSQL (users, cases, evidence metadata, trials, events, reports)
  +--> Private object storage (PDFs and uploaded files)
  +--> Qdrant (public legal chunks + separately scoped private document chunks)
  +--> Firecrawl (web discovery/extraction, then verification)
  +--> Background workers (parsing, OCR, indexing, long-running research)
  |
  v
LangGraph workflows
  |-- Case preparation graph
  |-- Trial graph: deterministic state + role-specific agent nodes
  |-- Report/evaluation graph
  |
  +--> Judge AI
  +--> Prosecutor AI
  +--> Defense AI
  +--> Witness AI (optional)
  +--> Legal retrieval and citation verification

Critical rules:
- The controller owns the process. Agents propose content; validated application
  logic decides what is allowed, what gets persisted, and who speaks next.
- A trial can contain multiple time-limited hearing sessions. Each session expires
  server-side, produces a report, and saves a continuation checkpoint.
- Nyaya Saathi is a private on-demand coach, not a courtroom speaker. Only user-
  submitted text becomes the user's courtroom contribution.
```

Build order to remember: product rules → data/security foundation → case workspace → narrow legal RAG → deterministic trial engine → agents → text streaming → evaluation/report → hardening. This order reduces rework and makes failures easier to diagnose.